Recurrence Relations

Rosen 8th ed., § 8.1-8.4

Recurrence Relations
(递推关系)

8 Advanced Counting Techniques

- 8.1 Recurrence Relations
- 8.2 **Solving Recurrence Relations**
  - Linear homogeneous recurrence relations with constant coefficients of degree k
    （k阶齐次常系数线性递推关系）
- 8.3 Divide-and-Conquer Algorithms and Recurrence Relations
- **8.4 Generating Functions**
- 8.5 Inclusion-Exclusion
- 8.6 Applications of Inclusion-Exclusion

§ 8.1: Recurrence Relations

- A *recurrence relation* (R.R., or just *recurrence*) for a sequence $\{a_n\}$ is an equation that expresses a_n in terms of one or more previous elements
  $a_0, \ldots, a_{n-1}$ of the sequence, for all $n \geq n_0$.
  - A recursive definition, without the base cases.
- A particular sequence (described non-recursively) is said to *solve* the given recurrence relation if it is consistent with the definition of the recurrence.
  - A given recurrence relation may have many solutions.

Recurrence Relation Example

- Consider the recurrence relation

  $a_n = 2a_{n-1} - a_{n-2}$ $(n \geq 2)$.

- Which of the following are solutions?

  $a_n = 3n$ Yes

  $a_n = 2^n$ No

  $a_n = 5$ Yes

Example Applications

- Recurrence relation for growth of a bank account with P% interest per given period:

  $$M_n = M_{n-1} + (P/100)M_{n-1}$$

- Growth of a population in which each organism yields 1 new one every period starting 2 periods after its birth.

  $$P_n = P_{n-1} + P_{n-2}$$ (Fibonacci relation)

Solving Compound Interest RR

- $M_n = M_{n-1} + (P/100)M_{n-1}$

  $= (1 + P/100)\, M_{n-1}$

  $= r\, M_{n-1}$ (let $r = 1 + P/100$)

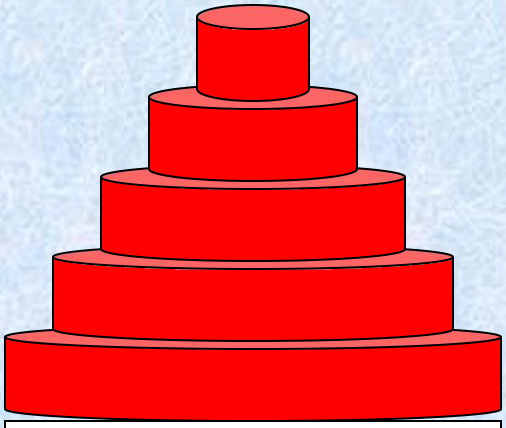
  $= r\,(r\, M_{n-2})$

  $= r \cdot r \cdot (r\, M_{n-3})$ …and so on to…

  $= r^n\, M_0$

Tower of Hanoi Example

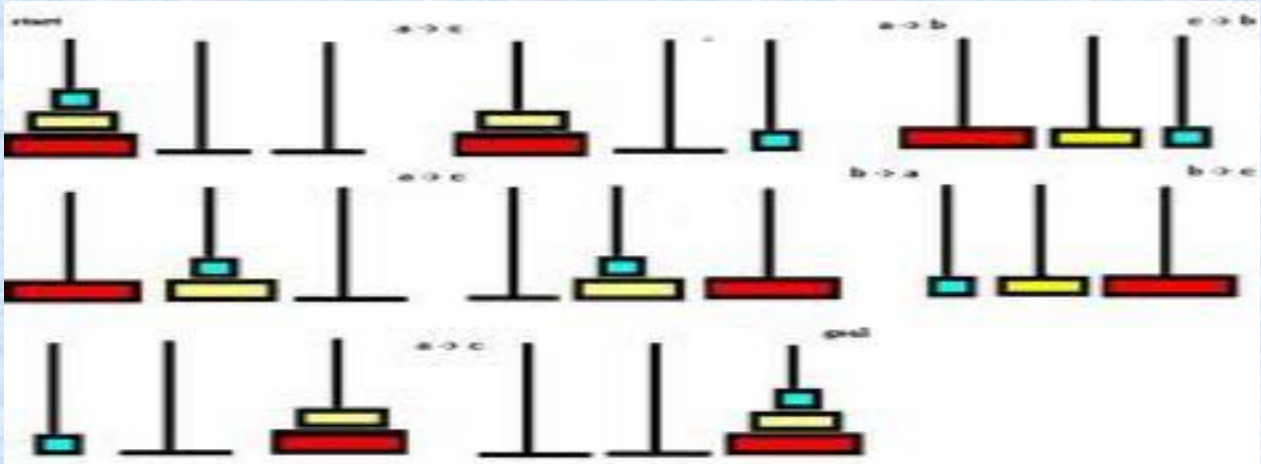
- Problem: Get all disks from peg 1 to peg 2.
  - Only move 1 disk at a time.
  - Never set a larger disk on a smaller one.



Peg #1 | Peg #2 | Peg #3

Hanoi汉诺塔问题

- 有三个木桩A，B，C，在A木桩上有n个大小不等的圆盘，按照由小到大的次序叠放，最大的最下，圆盘呈塔形。现需将圆盘逐个移到B木桩，并仍呈塔形，转移时可用C木桩，但要求在保持让较大的圆盘在较小的下面。问：完成这样的一次转移必须移动圆盘多少次？

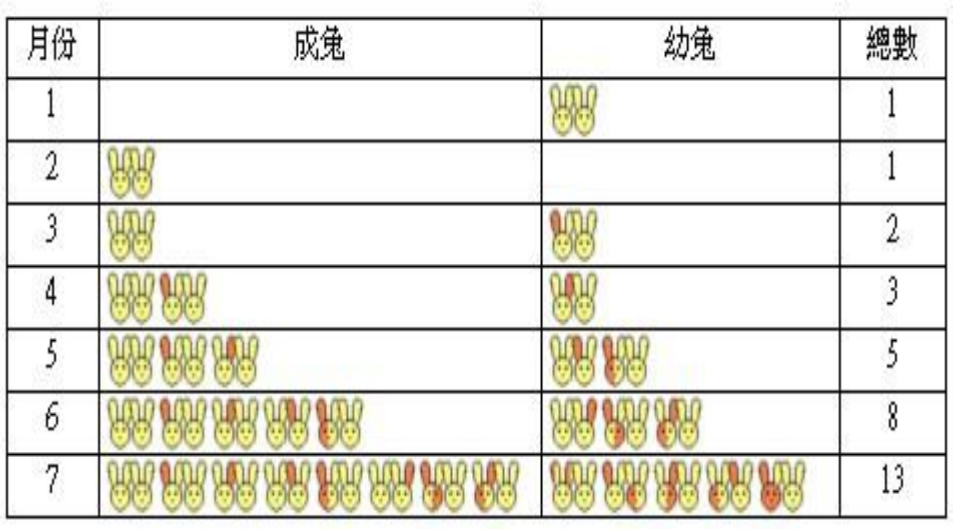


费波那契递推关系

- 13世纪意大利数学家(Fibonaci)提出一个有趣的兔子繁殖问题：在一年初把一对刚出生的小兔子放进养殖场。小兔满二个月后即可生育一对一雌一雄的小兔。以后每隔一个月即可生育一对一雌一雄的小兔。月复一月，问一年后养殖场有多少对兔子？

F（n）第n个月兔子对数

- F（0）＝0
- F（1）＝1
- F（2）＝1
- F（3）＝1＋1＝2
- F（4）＝2＋1＝3（三月数＋二月后代）
- F（5）＝3＋2＝5……
- F（13）＝144＋89＝233



月份	成兔	幼兔	總數
1			1
2			1
3			2
4			3
5			5
6			8
7			13

F(0)=0

F(1)=1

F(n)=F(n-1)+F(n-2)

费波那契递推关系

Example 6

- Find a recurrence relation and give initial conditions for the number of bit strings of length N that do not have two consecutive 0s. How many such bit strings are there of length five?

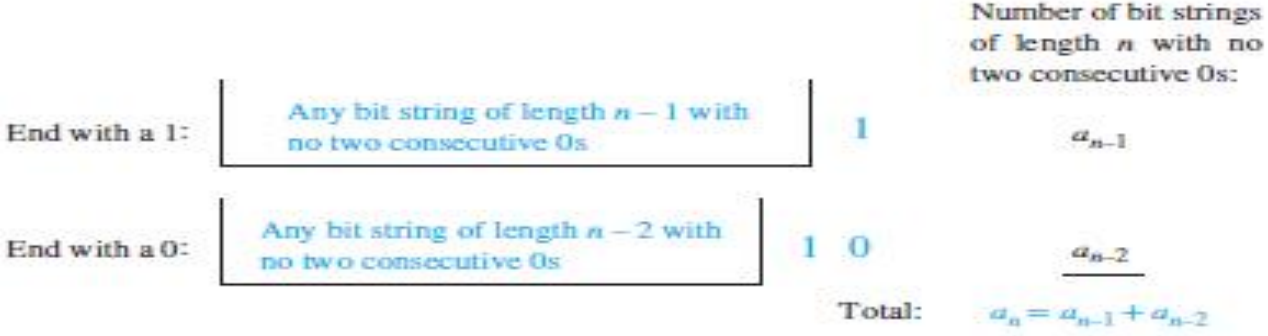


FIGURE 4 **Counting bit strings of length n with no two consecutive 0s.**

Example 7

- A computer system considers a string of decimal digits a valid codeword if it contains an even number of 0 digits. For instance,1230407869 is valid, whereas 120987045608 is not valid. Let a_n be the number of valid n-digit codewords. Find a recurrence relation for a_n.

there are

$$a_n = 9a_{n-1} + (10^{n-1} - a_{n-1})$$
$$= 8a_{n-1} + 10^{n-1}$$

valid strings of length n.

Hanoi Recurrence Relation

- Let H_n = # moves for a stack of n disks.
- Optimal strategy:
  - Move top $n-1$ disks to spare peg. (H_{n-1} moves)
  - Move bottom disk. (1 move)
  - Move top $n-1$ to bottom disk. (H_{n-1} moves)
- Note: $H_n = 2H_{n-1} + 1$

Solving Tower of Hanoi RR

$H_n = 2\,H_{n-1} + 1$

$= 2\,(2\,H_{n-2} + 1) + 1 \qquad = 2^2\,H_{n-2} + 2 + 1$

$= 2^2(2\,H_{n-3} + 1) + 2 + 1 \quad = 2^3\,H_{n-3} + 2^2 + 2 + 1$

…

$= 2^{n-1}\,H_1 + 2^{n-2} + \ldots + 2 + 1$

$= 2^{n-1} + 2^{n-2} + \ldots + 2 + 1$ (since $H_1 = 1$)

$= \sum_{i=0}^{n-1} 2^i$

$= 2^n - 1$ (From $\sum_{j=0}^{n} ar^j = \frac{ar^{n+1} - a}{r-1}$, where a=1, r=2)

Catalan 数

Find a recurrence relation for Cn, the number of ways to parenthesize the product of n+1 numbers, $x_0, x_1, x_2, \ldots, x_n$, to specify the order of multiplication. For example $C_3=5$ since there are five ways to parenthesize x_0, x_1, x_2, x_3, to to determine the order of multiplication.

$((x_0*x_1)*x_2)*x_3$ $(x_0*(x_1*x_2))*x_3$
$(x_0*x_1)*(x_2*x_3)$ $x_0*((x_1*x_2)*x_3)$ $x_0*(x_1*(x_2*x_3))$

$$C_n = C_0C_{n-1} + C_1C_{n-2} + \cdots + C_{n-2}C_1 + C_{n-1}C_0$$

$$= \sum_{k=0}^{n-1} C_kC_{n-k-1}.$$

Note that the initial conditions are $C_0 = 1$ and $C_1 = 1$.

Catalan 数

这一节讨论Catalan数,其递推关系是非线性的,许多有意义的计数问题都导致这样的递推关系.本节将举出一些,后面还将见到.

一个凸n边形,通过不相交于n边形的对角线,把n边形拆分成若干三角形,不同拆分的数目用 h_n 表之.

Catalan 数

例如五边形有如下五种拆分方案，故 $h_n = 5.$

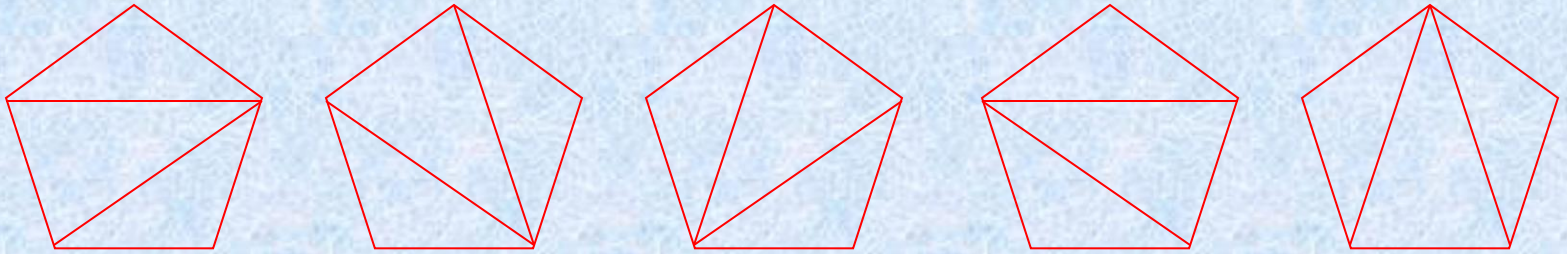


图 9.11-1

Catalan 数

1.递推关系

定理：

$$(a)\ h_{n+1} = h_2 h_n + h_3 h_{n-1} + \cdots + h_n h_2, \quad (2-11-1)$$

$$(b)\ (n-3)h_n = \frac{n}{2}(h_3 h_{n-1} + h_4 h_{n-2} + \cdots + h_{n-2} h_4 + h_{n-1} h_3). \quad (2-11-2)$$

Catalan 数

证明： (a) 的证明：

如图 $2-11-1$ 所示，以 v_1v_{n+1} 作为一个边的三角形 $v_1v_kv_{n+1}$，将凸 $n+1$ 边形分割成两部分，一部分是 k 边形，

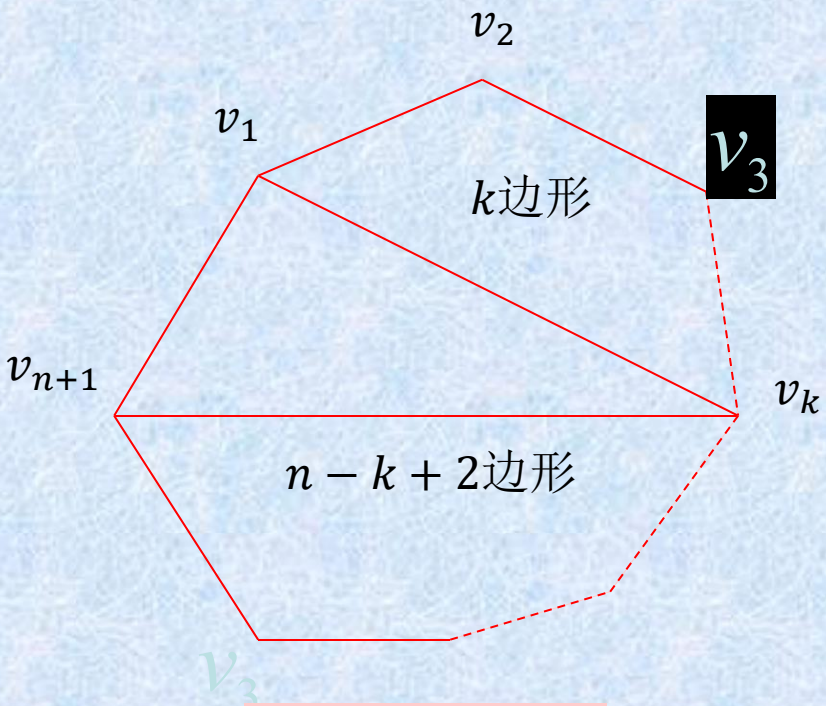


图 9.11-2

Catalan 数

另一部分是 $n-k+2$ 边形，$k=2,3,\cdots,n,$ 即 v_k 点可以是 $v_2,v_3,\cdots,v_n$ 点中任意一点。依据加法法则有

$$h_{n+1}=\sum_{k=2}^{n}h_k h_{n-k+2}$$
$$=h_2h_n+h_3h_{n-1}+\cdots+h_{n-1}h_3+h_nh_2.$$

Catalan 数

(b)的证明：

如图 $2-11-3$ 所示，从 (b) 点向其它 $n-3$个顶点 $\{v_3, v_4, \cdots, v_{n-1}\}$ 可引出 $n-3$ 条对角线。对角线 v_1v_k 把 n 边形分割成两个部分，因此

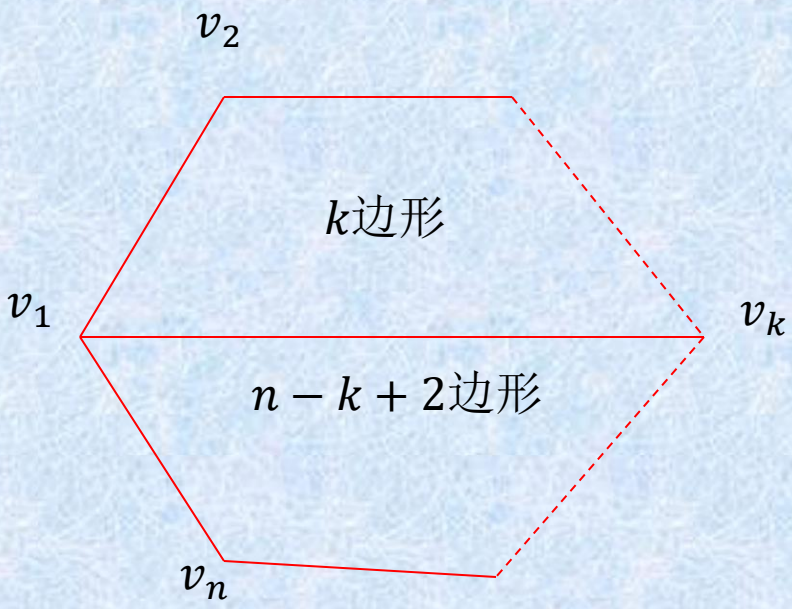


图 9.11-3

Catalan 数

以 v_1v_k 对角线作为拆分线的方案数为 h_kh_{n-k+2}

v_k 可以是 $v_3, v_4, \cdots, v_{n-1}$ 中任一点，对所有这些点求和得

$$h_3h_{n-1} + h_4h_{n-2} + \cdots + h_{n-2}h_4 + h_{n-1}h_3.$$

以 $v_2, v_3, \cdots, v_n$ 取代 v_1 点也有类似的结果。但考虑到对角线有两个顶点，同一对角线在两个顶点分别计算了一次，

Catalan 数

作

$$\frac{n}{2}(h_3h_{n-1}+h_4h_{n-2}+\cdots+h_{n-2}h_4+h_{n-1}h_3),(2-11-3)$$

$(2-11-3)$ 式并不就给出剖分数，无疑其中是有重复的。其重复度是由于一个凸 n 边形的剖分有 $n-3$ 条对角线，而对其每一条边计数时该剖分都计数了一次，故重复了 $n-3$ 次即 $2-11-3$ 式给出的结果是 h_n 的 $n-3$ 倍。

Catalan 数

$$\therefore \quad (n-3)h_n = \frac{n}{2}(h_3 h_{n-1} + h_4 h_{n-2} + \cdots + h_{n-2}h_4 + h_{n-1}h_3).$$

$(2-11-1)$ 式和 $(2-11-2)$式都是非线性的递推关系。

Catalan 数

2.Catalan 数计算公式

由 $(2-11-1)$ 式及 $h_2=1$,故得

$$h_{n+1}-2h_n=h_3h_{n-1}+h_4h_{n-2}+\cdots+h_{n-2}h_4+h_{n-1}h_3.$$

$$\therefore\quad (n-3)h_n=\frac{n}{2}(h_3h_{n-1}+h_4h_{n-2}+\cdots+h_{n-1}h_3)$$

$$=\frac{n}{2}(h_{n+1}-2h_n)$$

Catalan 数

由 $(n-3)h_n = \frac{n}{2}(h_{n+1} - 2h_n),$

整理得 $2(n-3)h_n = nh_{n+1} - 2nh_n.$

$$\therefore \quad nh_{n+1} = (4n-6)h_n.$$

令 $f_{n+1} = nh_{n+1},$

$$\therefore \quad f_{n+1} = (4n-6)\frac{f_n}{n-1} = \frac{2(n-3)}{n-1}f_n$$
$$= \frac{(2n-2)(2n-3)}{(n-1)(n-1)}f_n.$$

Catalan 数

即

$$\frac{f_{n+1}}{f_n}=\frac{(2n-2)(2n-3)}{(n-1)(n-1)},f_2=h_2=1,$$

$$\begin{aligned}f_{n+1}&=\frac{f_{n+1}}{f_n}\cdot\frac{f_n}{f_{n-1}}\cdot\frac{f_{n-1}}{f_{n-2}}\cdots\frac{f_4}{f_3}\cdot\frac{f_3}{f_2}\\&=\frac{(2n-2)(2n-3)}{(n-1)(n-1)}\cdot\frac{(2n-4)(2n-5)}{(n-2)(n-2)}\\&\cdots\frac{4\cdot3}{2\cdot2}\cdot\frac{2\cdot2}{1\cdot1}\end{aligned}$$

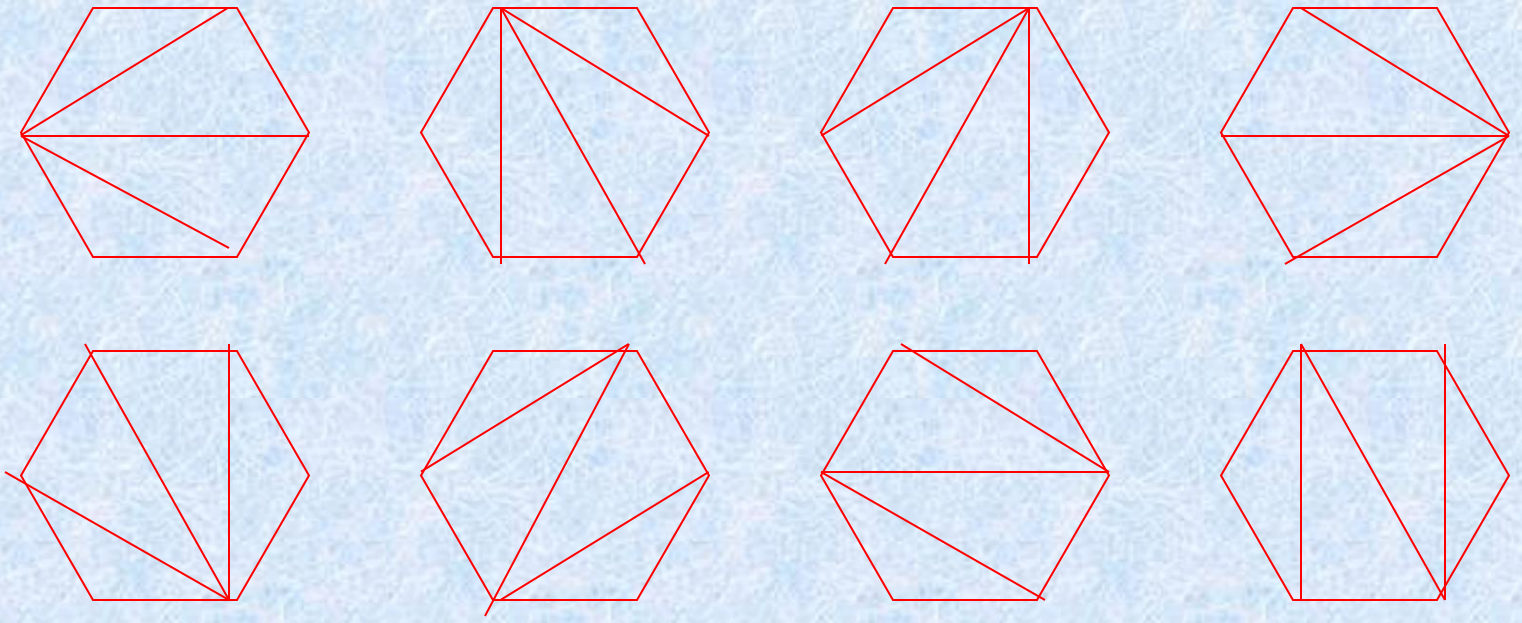
Catalan 数

$$= \frac{(2n-2)!}{(n-1)!\,(n-1)!} = \binom{2n-2}{n-1} = nb_{n+1}.$$

$$\therefore \quad h_{n+1} = \frac{1}{n}\binom{2n-2}{n-1}.$$

Catalan 数

4.举例



Catalan 数

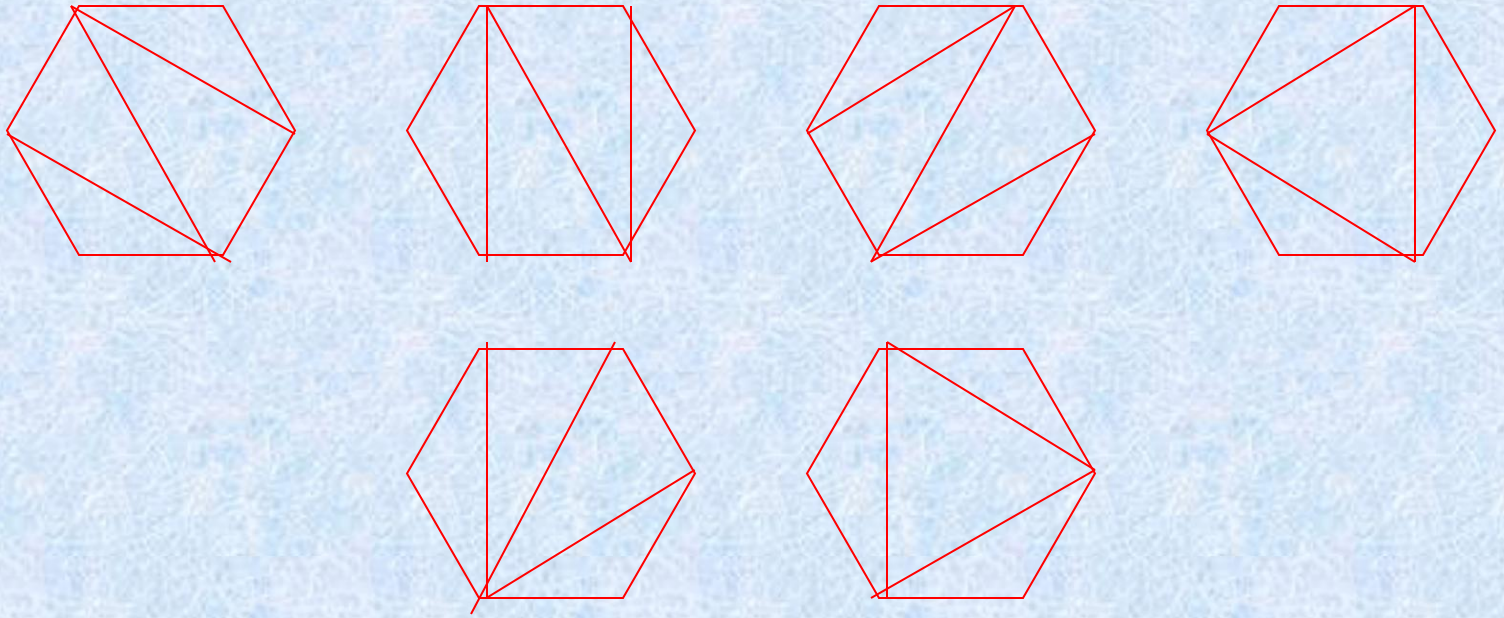


图 9.11-4

Catalan 数

例1. $h_6 = \frac{1}{5}\binom{8}{4} = 14$ ，见图 $11-4$

例2. $P = a_1 a_2 a_3 \cdots a_n$ 为n个数 $a_1, a_2, \cdots, a_n$ 的乘积，依据乘法的结合率，不改变其顺序，只用括号表示成对的乘积．试问有几种不同的乘法方案？

Catalan 数

令 p_n 表示n个数乘积的 $n-1$ 对括号插入的不同方案数.

$$p_n = p_1 p_{n-1} + p_2 p_{n-2} + \cdots + p_{n-1} p_1, p_1 = p_2 = 1.$$

令 $p_k = p_{k+1}, k = 1,2,3,\cdots,n,$ 故得

$$h_{n+1} = h_2 h_n + h_3 h_{n-1} + \cdots + h_{n-1} h_3 + h_n h_2,$$

而且 $h_{n+1} = h_{n+1} = 1.$ 故 p_n 即为Catalan数 $h_{n+1}.$

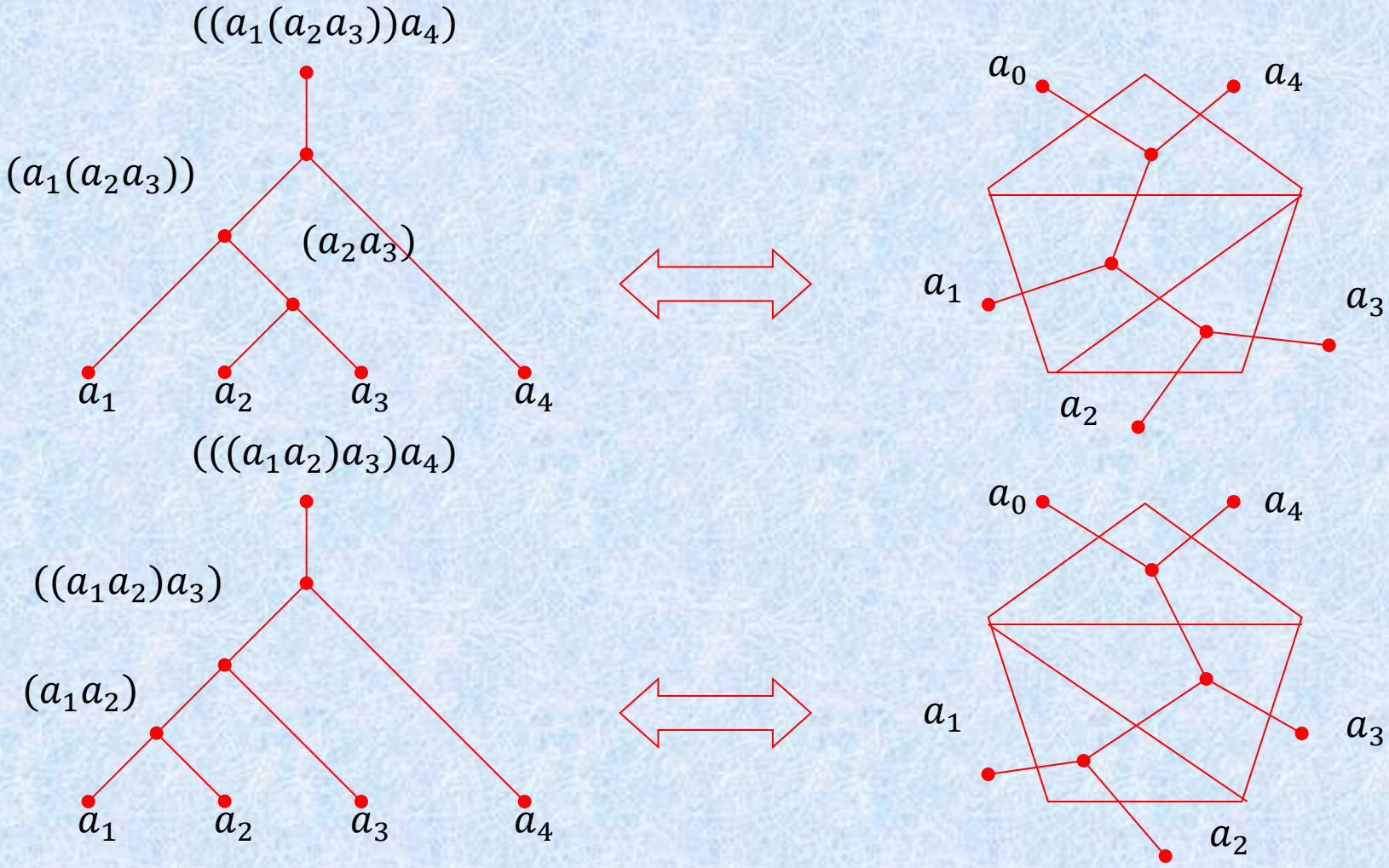
Catalan 数

以 $n-4$ 为例 $p_4 = h_5 = \frac{1}{4}\binom{6}{3} = 5.$

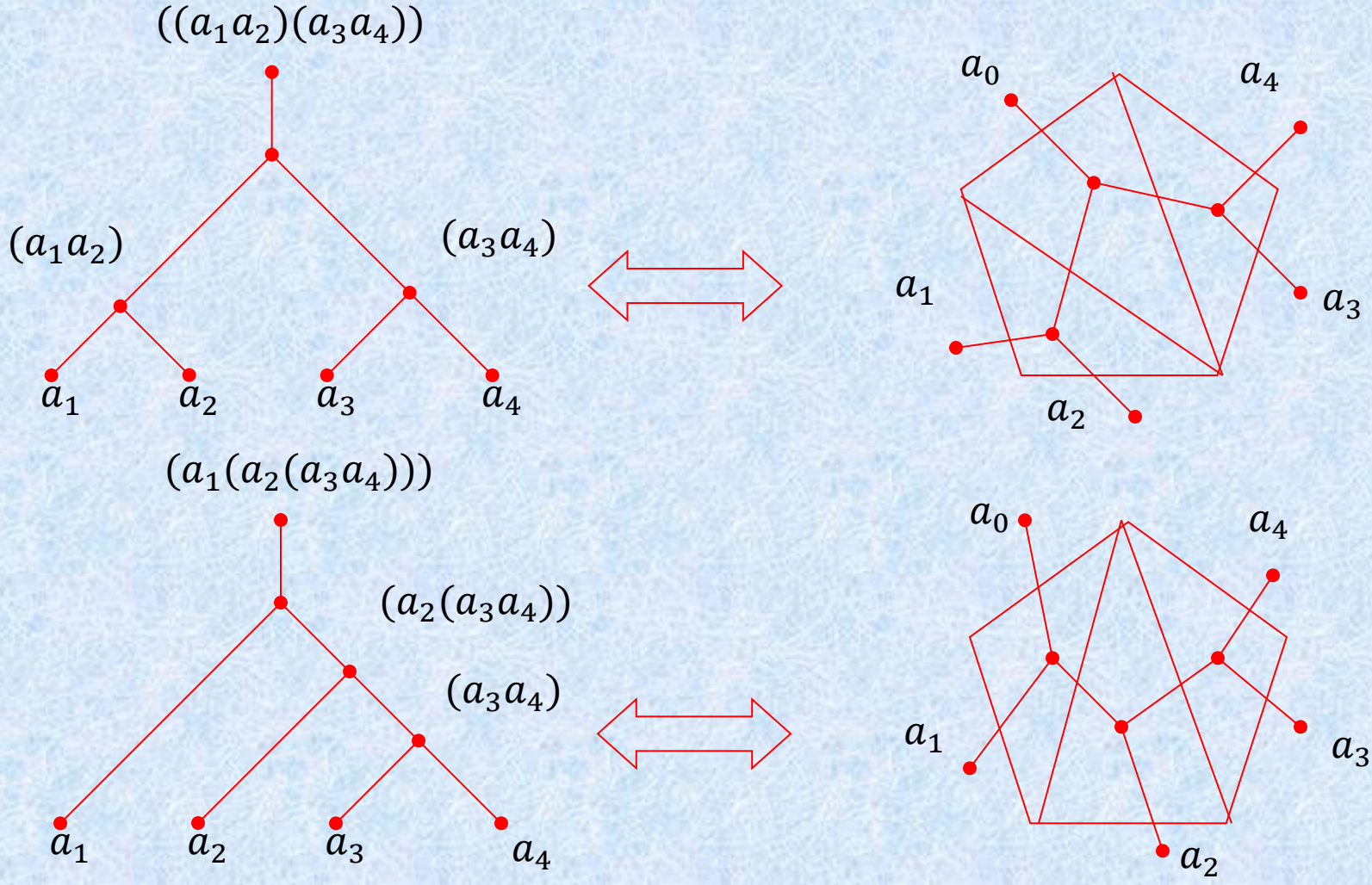
$$((a_1(a_2a_3))a_4), (((a_1a_2)a_3)a_4), ((a_1a_2)(a_3a_4)),$$
$$(a_1(a_2(a_3a_4))), (a_1((a_2a_3)a_4)). \quad (2-11-4)$$

$p_4 =$ Catalan数 h_5 ，下面建立 $(2-11-4)$ 式中不同的乘法顺序和一个5边形不同拆分的一一对应关系，如图 $2-11-6$

Catalan 数


$((a_1(a_2a_3))a_4)$
$(a_1(a_2a_3))$
(a_2a_3)
a_1
a_2
a_3
a_4
a_0
a_4
a_1
a_3
a_2
$(((a_1a_2)a_3)a_4)$
$((a_1a_2)a_3)$
(a_1a_2)
a_1
a_2
a_3
a_4
a_0
a_4
a_1
a_3
a_2

Catalan 数


$((a_1a_2)(a_3a_4))$
(a_1a_2)
(a_3a_4)
a_1
a_2
a_3
a_4
a_0
a_4
a_1
a_3
a_2
$(a_1(a_2(a_3a_4)))$
$(a_2(a_3a_4))$
(a_3a_4)
a_1
a_2
a_3
a_4
a_0
a_4
a_1
a_3
a_2

Catalan 数

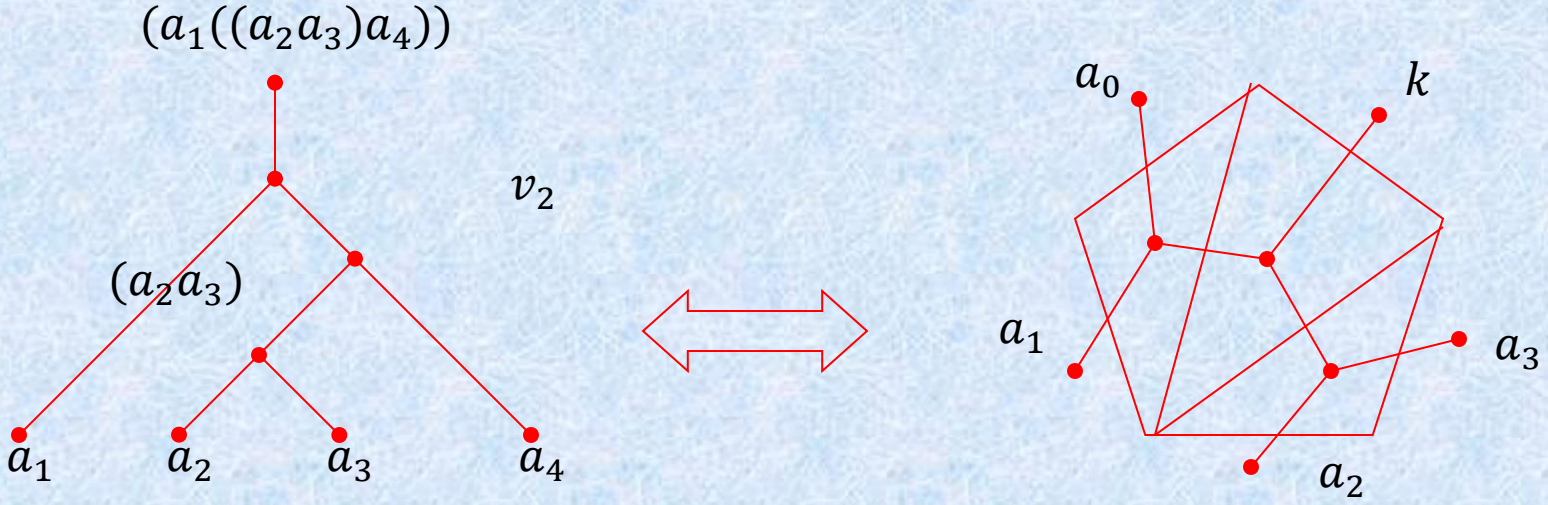
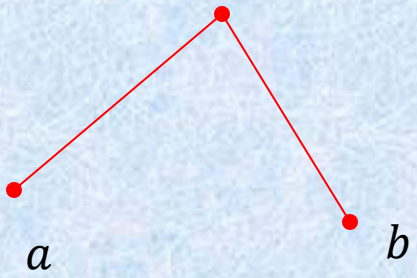


图 9.11-6

Catalan 数

$a \times b$ 运算用二分树表示，两片叶子分别表乘数和被乘数，分支点为运算符；如图 $n-k+2$边形

(b)



$(a \times b)$

图 9.11-5

Catalan 数

例3．n个1和n个0组成一2n位的2进制数，要求从左到右扫描，1的累计数不小于0的累计数，试求满足这条件的数有多少？

下面介绍两种算法。

解法1．设p_{2n} 为这样所得的数的个数。在2n

位上填入n个1的方案数为 $\binom{2n}{n}$ ，不填1的其余

Catalan 数

n位自动填以数0。从 $\binom{2n}{n}$ 中减去不符合要求

的方案数即为所求。不合要求的数指的是从左而右扫描，出现0的累计数超过1的累计数的数。

不合要求的数的特征是从左而右扫描时，必然在某一奇数 $2m+1$ 位上首先出现 $m+1$

Catalan 数

个0的累计数，和m个1的累计数。

此后的 h_n 位上有 个1，$n-m-1$ 个0。如若把后面这部分 $2(n-m)-1$ 位，0与1交换，使之成为 $n-m$ 个0，$n-m-1$ 个1，结果得1个由 $n+1$ 个0和 $n-1$ 个1组成的2n位数，即一个不合要求的数对应于一个由 $n-1$ 个0和 $n+1$ 个1组成的一个排列。

Catalan 数

反过来，任何一个由 $n+1$ 个0，$n-1$ 个1组成的2n位数，由于0的个数多2个，2n是偶数，故必在某一个奇数位上出现0的累计数超过1的累计数。同样在后面的部分，令0和1互换，使之成为由n个0和n个1组成的2n位数。即 $n+1$ 个0和 $n-1$ 个1组成的2n位数，必对应于一个不合要求的数。

Catalan 数

用上述方法建立了由 $n+1$ 个0和 $n-1$ 个1组成的2n位数，与由n个0和n个1组成的2n位数中从左向右扫描出现0的累计数超过1的累计数的数一一对应。

例如 $10\overset{*}{1}0101$ 是由4个0和4个1组成的8位2进制数。但从左而右扫描在第5位（打*号）出现0的累计数3超过1的累计数2，它对应于

Catalan 数

由3个1，5个0组成的 10100010 。

反过来 10100010^{*} 对应于 10100101 。

因而不合要求的2n位数与 $n+1$ 个0，$n-1$ 个1组成的排列一一对应，故有

$$p_{2n}=\binom{2n}{n}-\binom{2n}{n+1}=(2n)!\left[\frac{1}{n!\,n!}-\frac{1}{(n-1)!\,(n+1)!}\right]$$

Catalan 数

$$= (2n)!\left[\frac{n+1}{(n+1)!\,n!} - \frac{n}{(n+1)!\,n!}\right] = \frac{(2n)!}{(n+1)!\,n!}$$
$$= \frac{1}{n+1}\binom{2n}{n}.$$

Catalan 数

解法2.这个问题可以一一对应于图 2－11－7 中从原点 $(0,0)$ 到 (n,n) 点的路径要求中途所经过的点 (a,b) 满足关系 $a \leq b$

对应的办法是从 $(0,0)$ 出发，对2n位数从左而右扫描，若遇到1便沿y轴正方向走一格；若遇到0便沿x轴正方向走一格。由于有n个0，n个1，故对应一条从 $(0,0)$ 点到达 (n,n) 点的路

Catalan 数

径，由于要求1的累计数不少于0的累计数，故可以途经对角线 OA 上的点，但不允许穿越过对角线。反过来，满足这条件的路径对应一满足要求的2n位2进制数。见图 2－11－8

问题导致求从 $(0,0)$ 出发，途经对角线及对角线上方的点到达 (n,n) 点的路径数。

Catalan 数

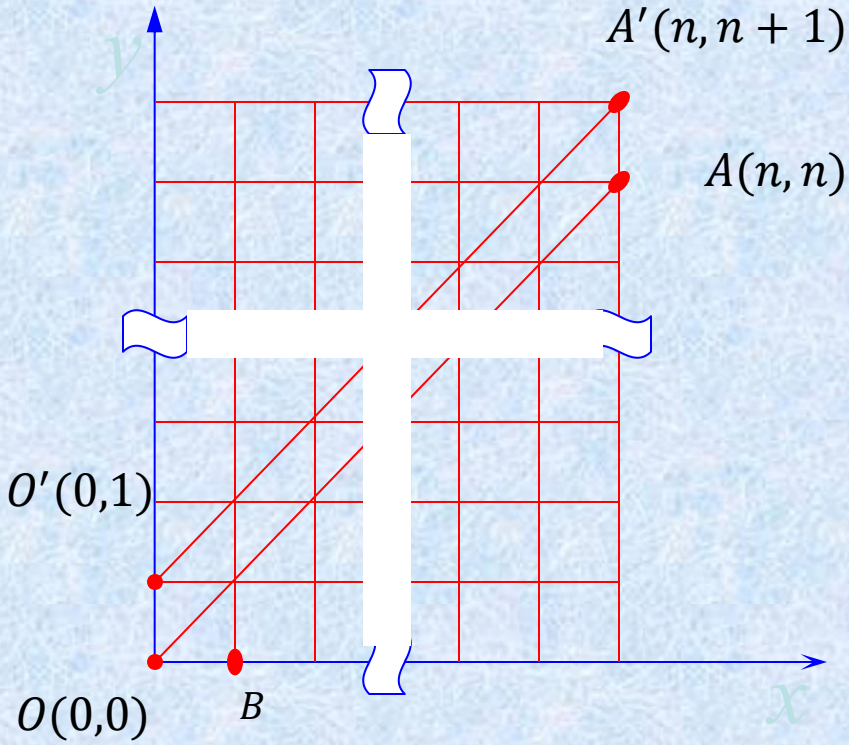


图 9.11-7

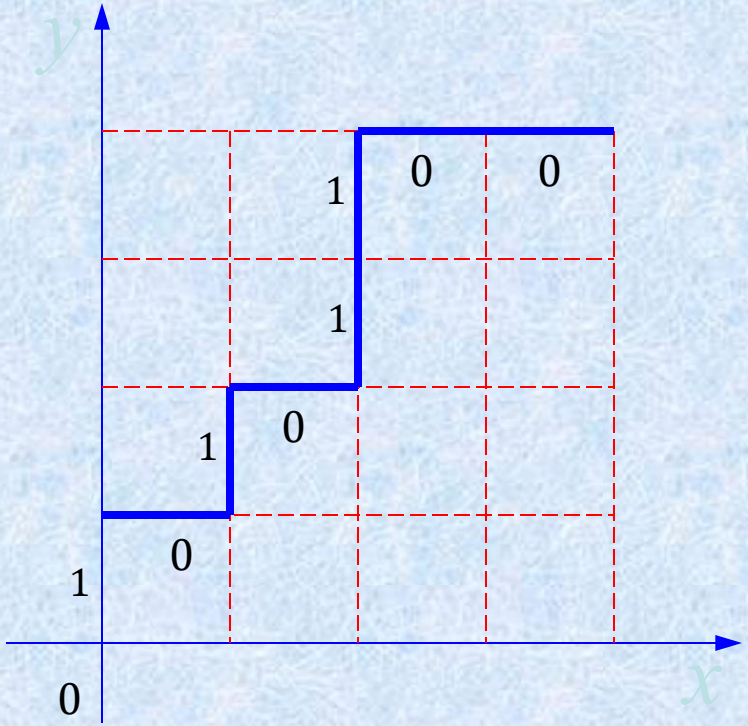


图 9.11-8

Catalan 数

从一点到另一点的路径数的讨论见第1章§3例1。见图 2－11－7 ，从 O 点出发经过 OA 及 OA 上方的点到达 A 点的路径对应一条从 O' 点出发经过 $O'A'$ 及 $O'A'$ 上方的点到达 A' 点的路径，这是显而易见的。

从 O 点出发途经 OA 上的点到达 A 点的路径，故对应一点从 O' 点出发穿越 $O'A'$ 到达 A'

Catalan 数

点的路径，故对应一从 O 点出发穿越 OA 到达 A 点的路径。

所以从 O 点出发经过 OA 及 OA 以上的点最后到达 A 点的路径树，等于从 O 点出发到达 A' 点的所有路径数，减去从 O' 点出发路径 OA 上的点到达 A 点的路径数。即

Catalan 数

$$p_{2n} = \binom{2n}{n} - \binom{2n}{n+1} = (2n)!\left[\frac{1}{n!\,n!} - \frac{1}{(n-1)!\,(n+1)!}\right]$$

$$= (2n)!\left[\frac{n+1}{(n+1)!\,n!} - \frac{n}{(n+1)!\,n!}\right] = \frac{(2n)!}{(n+1)!\,n!}$$

$$= \frac{1}{n+1}\binom{2n}{n} = h_{n+2}.$$

Catalan 数

例4. 由n个1，n个0组成的2n位二进制数，要求从左向右扫描前 位时1的累计数大于0的累计数，求满足这样条件的数的个数。此问题可归结为图 中从 点出发只经过对角线 上方的点抵达 点，求这样的路径数。相当于求从 点不经过对角线 ，抵达 点的路径数，于是便转换为

$2n-1$

$2n-1$

O

A

A

$O'(n-1,n+1)$

$A'(n-1,n)$

Catalan 数

例3的问题。

根据例3的结果。从 $O'(0,1)$ 点通过 A 的点，以及 $O'A'$ 上方的点到达 $(n-1,n)$ 的路径数为

$$\binom{2n-2}{n-1}-\binom{2n-2}{n}=(2n-2)!\,[\frac{1}{(n-1)!\,(n-1)!}$$

$$-\frac{1}{n!\,(n-2)!}]=(2n-2)!\frac{n-n+1}{n!\,(n-1)!}$$

Algorithms and Recurrence Relations

- An algorithm follows the dynamic programming paradigm when it recursively breaks down a problem into simpler overlapping subproblems, and computes the solution using the solutions of the subproblems. Generally, recurrence relations are used to find the overall solution from the solutions of the subproblems

- . Dynamic programming has been used to solve important problems in such diverse areas as economics, computer vision, speech recognition, artificial intelligence, computer graphics,and bioinformatics.

动态规划的定义

动态规划的基本思想是把待求解的问题分解成若干个子问题，先求解子问题,然后再从这些子问题的解得到原问题的解，其中用动态规划分解得到的子问题往往不是互相独立的。

动态规划在查找有很多重叠子问题的情况的最优解时有效.它将问题重新组合成子问题。为了避免多次解决这些子问题，它们的结果都逐渐被计算并被保存,从简单的问题直到整个问题都被解决。

动态规划的定义

因此,动态规划保存递归时的结果因而不会在解决同样的问题时花费时间.动态规划只能应用于有最优子结构的问题。最优子结构的意思是局部最优解能决定全局最优解(对有些问题这个要求并不能完全满足，故有时需要引入一定的近似).简单地说，问题能够分解成子问题来解决。

AN EXAMPLE OF DYNAMIC PROGRAMMING The problem we use to illustrate dynamic programming is related to the problem studied in Example 7 in Section 3.1. In that problem our goal was to schedule as many talks as possible in a single lecture hall. These talks have preset start and end times; once a talk starts, it continues until it ends; no two talks can proceed at the same time; and a talk can begin at the same time another one ends. We developed a greedy algorithm that always produces an optimal schedule, as we proved in Example 12 in Section 5.1. Now suppose that our goal is not to schedule the most talks possible, but rather to have the largest possible combined attendance of the scheduled talks.

We formalize this problem by supposing that we have n talks, where talk j begins at time t_j, ends at time e_j, and will be attended by w_j students. We want a schedule that maximizes the total number of student attendees. That is, we wish to schedule a subset of talks to maximize the sum of w_j over all scheduled talks. (Note that when a student attends more than one talk, this student is counted according to the number of talks attended.) We denote by $T(j)$ the maximum number of total attendees for an optimal schedule from the first j talks, so $T(n)$ is the maximal number of total attendees for an optimal schedule for all n talks.

We first sort the talks in order of increasing end time. After doing this, we renumber the talks so that $e_1 \leq e_2 \leq \cdots \leq e_n$. We say that two talks are **compatible** if they can be part of the same schedule, that is, if the times they are scheduled do not overlap (other than the possibility one ends and the other starts at the same time). We define $p(j)$ to be largest integer i, $i < j$, for which $e_i \leq s_j$, if such an integer exists, and $p(j) = 0$ otherwise. That is, talk $p(j)$ is the talk ending latest among talks compatible with talk j that end before talk j ends, if such a talk exists, and $p(j) = 0$ if there are no such talks.

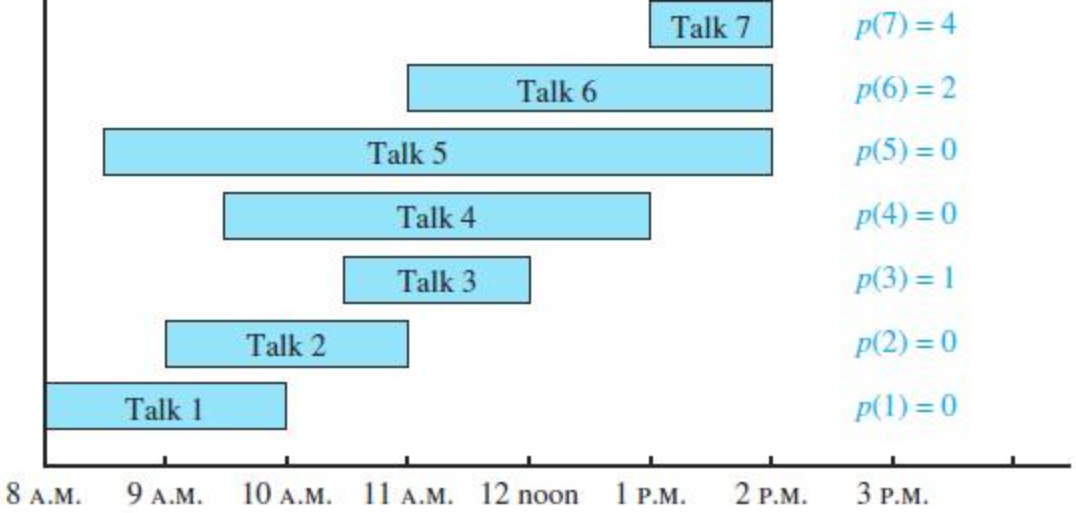


FIGURE 5 **A Schedule of Lectures with the Values of $p(n)$ Shown.**

EXAMPLE 6 Consider seven talks with these start times and end times, as illustrated in Figure 5.

Talk 1: start 8 A.M., end 10 A.M.
Talk 2: start 9 A.M., end 11 A.M.
Talk 3: start 10:30 A.M., end 12 noon
Talk 4: start 9:30 A.M., end 1 P.M.
Talk 5: start 8:30 A.M., end 2 P.M.
Talk 6: start 11 A.M., end 2 P.M.
Talk 7: start 1 P.M., end 2 P.M.

Find $p(j)$ for $j = 1, 2, \ldots, 7$.

Solution: We have $p(1) = 0$ and $p(2) = 0$, because no talks end before either of the first two talks begin. We have $p(3) = 1$ because talk 3 and talk 1 are compatible, but talk 3 and talk 2 are not compatible; $p(4) = 0$ because talk 4 is not compatible with any of talks 1, 2, and 3; $p(5) = 0$ because talk 5 is not compatible with any of talks 1, 2, 3, and 4; and $p(6) = 2$ because talk 6 and talk 2 are compatible, but talk 6 is not compatible with any of talks 3, 4, and 5. Finally, $p(7) = 4$, because talk 7 and talk 4 are compatible, but talk 7 is not compatible with either of talks 5 or 6.

To develop a dynamic programming algorithm for this problem, we first develop a key recurrence relation.

- $T(j) = \max(w_j + T(p(j)), T(j-1))$.

§ 8.2: Solving Recurrences

General Solution Schemas

- A *linear homogeneous recurrence of degree k with constant coefficients* ("*k*-LiHoReCoCo") is a recurrence of the form

  $$a_n = c_1 a_{n-1} + \ldots + c_k a_{n-k},$$

  where the c_i, *(i = 1, .., k)* are all real, and $c_k \neq 0$.
- The solution is uniquely determined if *k* initial conditions $a_0 \ldots a_{k-1}$ are provided.

Solving LiHoReCoCos

- Basic idea: Look for solutions of the form $a_n = r^n$, where r is a constant.
- This requires the *characteristic equation*:

  $r^n = c_1 r^{n-1} + \ldots + c_k r^{n-k}$, *i.e.*,

  $r^k - c_1 r^{k-1} - \ldots - c_{k-1} r - c_k = 0$
- The solutions (*characteristic roots*) can yield an explicit formula for the sequence.

Solving 2-LiHoReCoCos

- Consider an arbitrary 2-LiHoReCoCo:

  $a_n = c_1 a_{n-1} + c_2 a_{n-2}$

- It has the characteristic equation (C.E.):

  $r^2 - c_1 r - c_2 = 0$

- **Thm. 1:** If this CE has 2 roots $r_1 \neq r_2$, then

  $a_n = \alpha_1 r_1^n + \alpha_2 r_2^n$ for $n \geq 0$

  for some constants α_1, α_2.

Definition

- A recurrence relation is a linear homogeneous relation of degree k(k阶定常系数线性齐次递推关系） if it is of the form

$$a_n = r_1 a_{n-1} + r_2 a_{n-2} + \cdots + r_k a_{n-k}$$

with the r_i's constants.

- Note
  - On the right-hand side, the summands(被加数） are each built the same (homogeneous) way as a multiple of one of the k (degree k) previous terms (linear).
  - Keystone of this section.

Example 6

- (a) The relation $c_n=(-2)c_{n-1}$ is a linear homogeneous recurrence relation of degree 1.
- (b) The relation $a_n=a_{n-1} + 3$ is not a linear homogeneous recurrence relation.
- (c) The recurrence relation $f_n = f_{n-1}+ f_{n-2}$ is a linear homogeneous relation of degree 2.
- (') The recurrence relation $g_n = g_{n-1}^2+ g_{n-2}$ is not a linear homogeneous relation.

Characteristic equation (特征方程）

- For a linear homogeneous recurrence relation of degree k, $a_n = r_1 a_{n-1} + r_2 a_{n-2} + \cdots + r_k a_{n-k}$ We call the associated polynomial of degree k, $x^k = r_1 x^{k-1} + r_2 x^{k-2} + \ldots + r_k$ its characteristic equation (特征方程）.
- The roots of the characteristic equation play a key role in the explicit formula for the sequence defined by the recurrence relation and the initial conditions.

Theorem 1

- (a) If the characteristic equation $x^2 - r_1x - r_2 = 0$ of the recurrence relation $a_n = r_1a_{n-1} + r_2a_{n-2}$ has two distinct roots, s_1 and s_2, then $a_n = u\, s_1^n + v\, s_2^n$, where u and v depend on the initial conditions, is the explicit formula for the sequence.
- (b) If the characteristic equation $x^2 - r_1x - r_2 = 0$ has a single root s, the explicit formula is $a_n = u\, s^n + vns^n$, where u and v depend on the initial conditions.

Proof

- (a)
- Suppose that s_1 and s_2 are roots of $x^2 - r_1 x - r_2 = 0$, so $s_1^2 - r_1 s_1 - r_2 = 0$, $s_2^2 - r_1 s_2 - r_2 = 0$ and then $a_n = u\, s_1^n + v\, s_2^n$, for $n \geq 1$.
- We show that this definition of a_n defines the same sequence as $a_n = r_1 a_{n-1} + r_2 a_{n-2}$
- First we note that u and v are chosen so that $a_1 = u\, s_1 + v\, s_2$ and $a_2 = u\, s_1^2 + v\, s_2^2$ and so the initial conditions are satisfied. Then

$$
\begin{aligned}
a_n &= us_1^n + vs_2^n \\
&= us_1^{n-2}s_1^2 + vs_2^{n-2}s_2^2 \\
&= us_1^{n-2}(r_1 s_1 + r_2) + vs_2^{n-2}(r_1 s_2 + r_2) \\
&= r_1 us_1^{n-1} + r_2 us_1^{n-2} + r_1 vs_2^{n-1} + r_2 vs_2^{n-2} \\
&= r_1(us_1^{n-1} + vs_2^{n-1}) + r_2(us_1^{n-2} + vs_2^{n-2}) \\
&= r_1 a_{n-1} + r_2 a_{n-2}
\end{aligned}
$$

  - Q.E.D

- (b) omitted

Example

- Solve the recurrence $a_n = a_{n-1} + 2a_{n-2}$ given the initial conditions $a_0 = 2$, $a_1 = 7$.
- Solution: Use theorem 1
  - $c_1 = 1$, $c_2 = 2$
  - Characteristic equation:
    $r^2 - r - 2 = 0$
  - Solutions: $r = [-(-1) \pm ((-1)^2 - 4 \cdot 1 \cdot (-2))^{1/2}] / 2 \cdot 1$
    $= (1 \pm 9^{1/2})/2 = (1 \pm 3)/2$, so $r = 2$ or $r = -1$.
  - So $a_n = \alpha_1 2^n + \alpha_2 (-1)^n$.

Example Continued…

- To find α_1 and α_2, solve the equations for the initial conditions a_0 and a_1:

$$a_0 = 2 = \alpha_1 2^0 + \alpha_2 (-1)^0$$

$$a_1 = 7 = \alpha_1 2^1 + \alpha_2 (-1)^1$$

Simplifying, we have the pair of equations:

$$2 = \alpha_1 + \alpha_2$$

$$7 = 2\alpha_1 - \alpha_2$$

which we can solve easily by substitution:

$\alpha_2 = 2 - \alpha_1$; $7 = 2\alpha_1 - (2 - \alpha_1) = 3\alpha_1 - 2$;

$9 = 3\alpha_1$; $\alpha_1 = 3$; $\alpha_2 = -1$.

- Final answer: $a_n = 3 \cdot 2^n - (-1)^n$

Check: $\{a_{n\geq 0}\} = 2, 7, 11, 25, 47, 97 \ldots$

The Case of Degenerate Roots

- Now, what if the C.E. $r^2 - c_1 r - c_2 = 0$ has only 1 root r_0?
- **Theorem 2:** Then,

  $a_n = \alpha_1 r_0^n + \alpha_2 n r_0^n$, for all $n \geq 0$,

  for some constants α_1, α_2.

Example

- Find solution of recurrence relation

  $a_n = 6a_{n-1} - 9a_{n-2}$ with initial condition $a_0=1$, $a_1=6$

  The only root of $r^2 - 6r + 9 = 0$ is $r = 3$.

  Hence $a_n = \alpha_1 3^n + \alpha_2 n 3^n$

  From initial conditions, $a_0=1=\alpha_1$,

  $a_1=6=\alpha_1*3+\alpha_2*3$

  $\alpha_1 = 1$ and $\alpha_2 = 1$

  Therefore, $a_n = 3^n + n3^n$

k-LiHoReCoCos

- Consider a *k*-LiHoReCoCo:
- It's C.E. is:

$$a_n = \sum_{i=1}^{k} c_i a_{n-i}$$

$$r^k - \sum_{i=1}^{k} c_i r^{k-i} = 0$$

- **Thm.3:** If this has k distinct roots r_i, then the solutions to the recurrence are of the form:

$$a_n = \sum_{i=1}^{k} \alpha_i r_i^n$$

for all $n \geq 0$, where the α_i are constants.

Degenerate *k*-LiHoReCoCos

- Suppose there are t roots $r_1,\ldots,r_t$ with multiplicities $m_1,\ldots,m_t$. Then:

$$a_n = \sum_{i=1}^{t}\left(\sum_{j=0}^{m_i-1}\alpha_{i,j}n^j\right)r_i^n$$

for all $n\geq 0$, where all the α are constants.

Li**No**ReCoCos

- Linear *nonhomogeneous* RRs with constant coefficients may (unlike Li**Ho**ReCoCos) contain some terms $F(n)$ that depend *only* on n (and *not* on any a_i's). General form:

$$a_n = c_1 a_{n-1} + \ldots + c_k a_{n-k} + F(n)$$

The *associated homogeneous recurrence relation* (associated LiHoReCoCo).

Solutions of LiNoReCoCos

- A useful theorem about LiNoReCoCos:
  - If $a_n = p(n)$ is any *particular* solution to the LiNoReCoCo

$$a_n = \left(\sum_{i=1}^{k} c_i a_{n-i} \right) + F(n)$$

  - Then *all* its solutions are of the form:

$$a_n = p(n) + h(n),$$

    where $a_n = h(n)$ is any solution to the associated homogeneous RR

$$a_n = \left(\sum_{i=1}^{k} c_i a_{n-i} \right)$$

Example

- Find all solutions to $a_n = 3a_{n-1}+2n$. Which solution has $a_1 = 3$?
  - Notice this is a 1-LiNoReCoCo. Its associated 1-LiHoReCoCo is $a_n = 3a_{n-1}$, whose solutions are all of the form $a_n = \alpha 3^n$. Thus the solutions to the original problem are all of the form $a_n = p(n) + \alpha 3^n$. So, all we need to do is find one $p(n)$ that works.

Trial Solutions

- If the extra terms $F(n)$ are a degree-t polynomial in n, you should try a degree-t polynomial as the particular solution $p(n)$.
- This case: $F(n)$ is linear so try $a_n = cn + d$.

  $cn+d = 3(c(n-1)+d) + 2n$ (for all n)
  $(-2c-2)n + (3c-2d) = 0$ (collect terms)
  So $c = -1$ and $d = -3/2$.

  So $a_n = -n - 3/2$ is a solution.
- Check: $a_{n\geq 1} = \{-5/2, -7/2, -9/2, \ldots\}$

Finding a Desired Solution

- From the previous, we know that all general solutions to our example are of the form:

  $a_n = -n - 3/2 + \alpha 3^n$.

  Solve this for α for the given case, $a_1 = 3$:

  $3 = -1 - 3/2 + \alpha 3^1$

  $\alpha = 11/6$

- The answer is $a_n = -n - 3/2 + (11/6)3^n$

§ 8.3: Divide & Conquer R.R.s

Main points so far:

- Many types of problems are solvable by reducing a problem of size n into some number a of independent subproblems, each of size $\leq \lceil n/b \rceil$, where $a \geq 1$ and $b > 1$.
- The time complexity to solve such problems is given by a recurrence relation:
  - $T(n) = a\,T(\lceil n/b \rceil) + g(n)$

Time for each subproblem

Time to break problem up into subproblems

Binary Search

- Here is a description of the binary search algorithm in pseudocode.

procedure binary search(x: integer, $a_1, a_2, \ldots, a_n$: increasing integers)
$i := 1$ {i is the left endpoint of interval}
$j := n$ {j is right endpoint of interval}
while $i < j$
    $m := \lfloor (i + j)/2 \rfloor$
    if $x > a_m$ then $i := m + 1$
    else $j := m$
if $x = a_i$ **then** *location* $:= i$
else *location* $:= 0$
return *location*{location is the subscript i of the term a_i equal to x, or 0 if x is not found}

Divide+Conquer Examples

- **Binary search:** Break list into 1 sub-problem (smaller list) (so a=1) of size $\leq \lceil n/2 \rceil$ (so b=2).
  - So $T(n) = T(\lceil n/2 \rceil)+c$ ($g(n)=c$ constant)
- **Merge sort:** Break list of length n into 2 sublists (a=2), each of size $\leq \lceil n/2 \rceil$ (so b=2), then merge them, in $g(n) = \Theta(n)$ time.
  - So $T(n) = T(\lceil n/2 \rceil) + cn$ (roughly, for some c)

Binary Multiplication of Integers

- Algorithm for computing the product of two n bit integers.

procedure *multiply*(a, b: positive integers)
{the binary expansions of a and b are $(a_{n-1},a_{n-2},\ldots,a_0)_2$ and $(b_{n-1},b_{n-2},\ldots,b_0)_2$, respectively}
for $j := 0$ to $n-1$
    if $b_j = 1$ **then** $c_j = a$ shifted j places
    else $c_j := 0$
{$c_o,c_1,\ldots, c_{n-1}$ are the partial products}
$p := 0$
for $j := 0$ to $n-1$
    $p := p + c_j$
return p {p is the value of ab}

- The number of additions of bits used by the algorithm to multiply two n-bit integers is $O(n^2)$.

Fast Multiplication Example

- The ordinary grade-school algorithm takes $\Theta(n^2)$ steps to multiply two n-digit numbers.P225
  - This seems like too much work!
- So, let's find an asymptotically *faster* multiplication algorithm!

- To find the product cd of two $2n$-digit base-b numbers, $c=(c_{2n-1}c_{2n-2}\ldots c_0)_b$ and $d=(d_{2n-1}d_{2n-2}\ldots d_0)_b$, first, we break c and d in half:
  $c=b^nC_1+C_0$, $d=b^nD_1+D_0$,
  and then... (see next slide)

Derivation of Fast Multiplication

$$cd = (b^n C_1 + C_0)(b^n D_1 + D_0)$$

$$= b^{2n} C_1 D_1 + b^n (C_1 D_0 + C_0 D_1) + C_0 D_0$$

$$= b^{2n} C_1 D_1 + C_0 D_0 +$$

$$b^n (C_1 D_0 + C_0 D_1 + (C_1 D_1 - C_1 D_1) + (C_0 D_0 - C_0 D_0))$$

$$= (b^{2n} + b^n) C_1 D_1 + (b^n + 1) C_0 D_0 +$$

$$b^n (C_1 D_0 - C_1 D_1 - C_0 D_0 + C_0 D_1)$$

$$= (b^{2n} + b^n) C_1 D_1 + (b^n + 1) C_0 D_0 +$$

$$b^n (C_1 - C_0)(D_0 - D_1)$$

(Multiply out polynomials)

Zero

(Factor last polynomial)

Three multiplications, each with n-digit numbers

Recurrence Rel. for Fast Mult.

Notice that the time complexity $T(n)$ of the fast multiplication algorithm obeys the recurrence:

- $T(2n)=3T(n)+\Theta(n)$

  i.e.,

- $T(n)=3T(n/2)+\Theta(n)$

  So $a=3$, $b=2$.

Time to do the needed adds & subtracts of n-digit and $2n$-digit numbers

Matrix Multiplication Algorithm

- The definition for matrix multiplication can be expressed as an algorithm; **C** = **A B** where **C** is an $m \times n$ matrix that is the product of the $m \times k$ matrix **A** and the $k \times n$ matrix **B**.
- This algorithm carries out matrix multiplication based on its definition.

procedure *matrix multiplication*(**A**,**B***:* matrices)
  for *i* := 1 to *m*
    for *j* := 1 to *n*
      $c_{ij} := 0$
      for *q* := 1 to *k*
        $c_{ij} := c_{ij} + a_{iq}\, b_{qj}$
return C{**C = [**c_{ij}**]** is the product of **A** and **B**}

$\mathbf{A} = [a_{ij}]$ is a $m \times k$ matrix
$\mathbf{B} = [b_{ij}]$ is a $k \times n$ matrix

Complexity of Matrix Multiplication

Example: How many additions of integers and multiplications of integers are used by the matrix multiplication algorithm to multiply two $n \times n$ matrices.

Solution: There are n^2 entries in the product. Finding each entry requires n multiplications and $n - 1$ additions. Hence, n^3 multiplications and $n^2(n - 1)$ additions are used.

Hence, the complexity of matrix multiplication is $O(n^3)$.

Example 5

- Fast Matrix Multiplication

- Seven multiplication of two (n/2)*(n/2) matrices and 15 additions of (n/2)*(n/2) matrices.
- $f(n)=7f(n/2)+15n^2/4$

- *Let $n=b^k$, where k is a positive integer. then*
- $f(n)=af(n/b)+g(n)$
- $=a^2f(n/b^2)+ag(n/b)+g(n)$
- $= a^3f(n/b^3)+ a^2f(n/b^2)+ag(n/b)+g(n)$
- ……..
- $= a^kf(n/b^k)+\sum_{j=0}^{k-1}a^jg(n/b^j)$

Theorem 1

Let *f* be an increasing function that satisfies the recurrence relation:

$$f(n) = af(n/b) + c$$

with $a \geq 1$, integer $b > 1$, real $c > 0$, Then:

$$f(n) \quad is \quad \begin{cases} O(n^{\log_b a}) & \text{if } a > 1 \\ O(\log n) & \text{if } a = 1 \end{cases}$$

Furthermore, when $n=b^k$,for all $k \in Z+$,

$$f(n) = C_1 n^{\log_b a} + C_2$$

Where $C_1=f(1)+c/(a-1)$ and $C_2=-c(a-1)$

Example 6

- Let f(n)=5f(n/2)+3 and f(1)=7. Find f(2k) where *k* is a positive integer. Also, estimate *f(n)* if *f* is an increasing function.

- Binary search: f(n)=f(n/2)+2

The Master Theorem(主定理)

Consider a function $f(n)$ that, for all $n=b^k$ for all $k \in \mathbf{Z}^+$, satisfies the recurrence relation:

$$f(n) = af(n/b) + cn^d$$

with $a \geq 1$, integer $b>1$, real $c>0$, $d \geq 0$. Then:

$$f(n) \in \begin{cases} O(n^d) & \text{if } a < b^d \\ O(n^d \log n) & \text{if } a = b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Example 9

- Complexity of Merge sort
- M(n)=2M(n/2)+n
- Thus, a=2, b=2, d=1. So $a = b^d$, so case 2 of the master theorem applies, so:
- M(n) is $O(n \log n)$.

Master Theorem Example

- Recall that complexity of fast multiply was:

  $T(n)=3T(n/2)+\Theta(n)$

- Thus, a=3, b=2, d=1. So $a > b^d$, so case 3 of the master theorem applies, so:

  $$T(n) = O(n^{\log_b a}) = O(n^{\log_2 3})$$

  which is $O(n^{1.58\ldots})$, so the new algorithm is strictly faster than ordinary $\Theta(n^2)$ multiply!

Example 11

- Fast Matrix Multiplication

- Seven multiplication of two (n/2)*(n/2) matrices and 15 additions of (n/2)*(n/2) matrices.
- $f(n)=7f(n/2)+15n^2/4$

Example: Mergesort

```
list mergesort(list L)
{
    if (length(L) == 1) then return L;
    return merge(mergesort(first half of L),
                 mergesort(second half of L));
}
```

- Assume merge = $O(n)$. What is the running time of mergesort?

Recurrences

- $T(1) = 0$
- $T(n) = 2*T(n/2)+n$
- How do we solve this?
  - Generating functions

Guess and Check

- It's a divide and conquer algorithm, so think $\Theta(n \log n)$.
  - Let's guess $T(n) = n \log_2 n$
  - $T(1) = 0$. Check
  - $T(n) = 2\ T(n/2) + n$

    $= 2\ (n/2 \log_2(n/2)) + n$

    $= n \log_2(n/2) + n$

    $= n \log_2(n) - n \log_2 2 + n$

    $= n \log_2(n) - n + n$

    $= n \log_2(n)$

Guess the Form

- Guess: $T(n) = a\ n\ log_2(n) + b\ n + c$
  - $T(n) = a\ n\ log_2(n) + b\ n + c$

Guess the Form

- $T(1) = 0 = b + c$
- $T(n) = 2\ T(n/2) + n$

  $= 2\ (a\ (n/2)\ log_2\ (n/2) + b(n/2) + c) + n$

  $= a\ n\ log_2\ (n/2) + (b+1)\ n + 2c$

  $= a\ n\ log_2\ n - a\ n\ log_2\ 2 + (b+1)\ n + 2c$

  $= a\ n\ log_2\ n - a\ n + (b+1)\ n + 2c$

  Should = $a\ n\ log_2\ n + b\ n + c$
- Therefore, $b+1 - a = b \Rightarrow a = 1$. $2c = c \Rightarrow c = 0$. This leaves $b = 0$.

- $T(n) = n\ log_2\ (n)$

Recurrence relations

$T(1) = 0$

$T(n) = 2*T(n/2)+n$

$n=2^k \qquad k=0,1,2,\ldots$

$S(k)=2*S(k-1)+2^k \qquad S(0)=0, S(1)=2$

$S(k-1) = 2*S(k-2)+2^{k-1}$

$S(k)=4*S(k-1)-4*S(k-2)$

$S(k)=u2^k+vk2^k$

$u=0, v=1 \quad S(k)=k2^k,$

$T(n)=n \log_2 (n)$

8.4 Generating Functions.

Def 1. The generating function for the sequence $\{a_n\}$ is the infinite power series.

$$G(x) = a_0 + a_1 x + \ldots + a_n x^n + \ldots$$

$$= \sum_{k=0}^{\infty} a_k x^k$$

(若 $\{a_n\}$ 是finite，可视为是infinite，但后面的term都等于0)

Example 2. What is the generating function for the sequence 1,1,1,1,1,1 ?

Sol :

$$G(x) = 1 + x + x^2 + \ldots + x^5$$

(expansion)

$$= \frac{x^6 - 1}{x - 1}$$

(closed form)

Example 3.

Let $m \in \mathbf{Z}^+$ and $a_k = \binom{m}{k}$, for $k = 0, 1, \ldots, m$.

What is the generating function for the sequence $a_0, a_1, \ldots, a_m$?

Sol :

$$G(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_m x^m$$

$$= \binom{m}{0} + \binom{m}{1} x + \binom{m}{2} x^2 + \cdots + \binom{m}{m} x^m$$

$$= (1+x)^m \qquad \text{(by 二项式定理)}$$

Example 4.

The function $f(x) = \dfrac{1}{1 - x}$ is the generating

function of the sequence $1,1, 1, 1\ldots,$

since $\dfrac{1}{1 - x} = 1 + x + x^2 + \ldots = \sum_{k=0}^{\infty} x^k$

when $|x| < 1$.

Example 5.

The function $f(x) = \dfrac{1}{1-ax}$ is the generating function of the sequence $1, a, a^2, \ldots,$

since $\dfrac{1}{1-ax} = 1 + ax + a^2x^2 + \ldots = \sum_{k=0}^{\infty} (ax)^k$

when $|ax| < 1$ for $a \neq 0$

Theroem1

- Let $f(x)=\sum_{k=0}^{\infty} a_k x^k$ and $g(x)=\sum_{k=0}^{\infty} b_k x^k$. Then

$$f(x)+g(x)=\sum_{k=0}^{\infty}(a_k+b_k)x^k$$

- and

- $$f(x)g(x)=\sum_{k=0}^{\infty}\left(\sum_{j=0}^{k} a_j b_{k-j}\right)x^k$$.

Example 6

Let $f(x)=1/(1-x)^2$. Use Example 4

$1/(1-x)=1+x+x^2+x^3+\ldots$

to find the coefficients a_0, a_1, a_2,… in the expansion $f(x)=\sum_{k=0}^{\infty} a_k x^k$.

Def 2.

Let $u \in \mathbf{R}$ and $k \in \mathbf{Z}^+ \cup \{0\}$. Then the extended binomial coefficient $\binom{u}{k}$ is defined by

$$\binom{u}{k} = \begin{cases} \dfrac{1}{k!} u(u-1)(u-2)\ldots(u-k+1), & \text{if } k > 0 \\ 1 & \text{if } k = 0 \end{cases}$$

Example 7.

Find $\binom{-2}{3}$ and $\binom{\frac{1}{2}}{3}$

Sol :

$$\binom{-2}{3} = \frac{1}{3!}(-2)(-3)(-4) = -4$$

$$\binom{\frac{1}{2}}{3} = \frac{1}{3!}\left(\frac{1}{2}\right)\left(\frac{-1}{2}\right)\left(\frac{-3}{2}\right) = \frac{1}{16}$$

Thm 2. (The Extended Binomial Theorem)

Let $x \in \mathbf{R}$ with $|x|<1$ and let $u \in \mathbf{R}$, then

$$(1+x)^u = \sum_{k=0}^{\infty} \binom{u}{k} x^k$$

Example 9.

Find the generating functions for $(1+x)^{-n}$ and $(1-x)^{-n}$ where $n \in \mathbf{Z}^+$

Sol : By the Extended Binomial Theorem,

$$(1+x)^{-n} = \sum_{k=0}^{\infty} \binom{-n}{k} x^k = \sum_{k=0}^{\infty} \frac{1}{k!}(-n)(-n-1)\ldots(-n-k+1)x^k$$

$$= \sum_{k=0}^{\infty} \frac{(-1)^k}{k!}(n)(n+1)\ldots(n+k-1)x^k$$

$$= \sum_{k=0}^{\infty} (-1)^k \binom{n+k-1}{k} x^k$$

Note : $\binom{-n}{k} = (-1)^k \binom{n+k-1}{k}$

By replacing x by $-x$ we have

$$(1-x)^{-n} = \sum_{k=0}^{\infty} \binom{n+k-1}{k} x^k$$

P448
Theorem2

The following table lists a number of useful generating functions, many of which can be obtained from known formulas for sums of series.

Useful Generating Functions	
$G(x)$	a_k
$(1+x)^n = \sum_{k=0}^{n} C(n,k)x^k$	$C(n,k)$
$(1+ax)^n = \sum_{k=0}^{n} C(n,k)a^k x^k$	$C(n,k)a^k$
$(1+x^r)^n = \sum_{k=0}^{n} C(n,k)x^{rk}$	$C(n,k/r)$ if $r\|k$; 0 otherwise
$\frac{1-x^{n+1}}{1-x} = \sum_{k=0}^{n} x^k$	1 if $k \leq n$; 0 otherwise
$\frac{1}{1-x} = \sum_{k=0}^{\infty} x^k$	1
$\frac{1}{1-ax} = \sum_{k=0}^{\infty} a^k x^k$	a^k
$\frac{1}{1-x^r} = \sum_{k=0}^{\infty} x^{rk}$	1 if $r\|k$; 0 otherwise
$\frac{1}{(1-x)^2} = \sum_{k=0}^{\infty}(k+1)x^k$	$k+1$
$\frac{1}{(1-x)^n} = \sum_{k=0}^{\infty} C(n+k-1,k)x^k$	$C(n+k-1,k)$
$\frac{1}{(1+x)^n} = \sum_{k=0}^{\infty} C(n+k-1,k)(-1)^k x^k$	$(-1)^k C(n+k-1,k)$
$\frac{1}{(1-ax)^n} = \sum_{k=0}^{\infty} C(n+k-1,k)a^k x^k$	$C(n+k-1,k)a^k$
$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}$	$1/k!$
$\ln(1+x) = \sum_{k=0}^{\infty} \frac{(-1)^{k+1}}{k} x^k$	$(-1)^{k+1}/k$

※Counting Problems and Generating Functions

Example 10.

Find the number of solutions of

$e_1 + e_2 + e_3 = 17$

Where e_1, e_2 and e_3 are nonnegative integers with $2 \le e_1 \le 5$, $3 \le e_2 \le 6$ and $4 \le e_3 \le 7$.

Sol : the coefficient of x^{17}

$$(x^2 + x^3 + x^4 + x^5)(x^3 + x^4 + x^5 + x^6)(x^4 + x^5 + x^6 + x^7)$$

Example 13,14

- Use generating functions to find the number of k-combinations of a set with n elements. Assume the Binomial Theorem has already been established.

- Use generating functions to find the number of k-combinations of a set with n elements when repetition of elements is allowed.

※Using Generating Functions to solve Recurrence Relations.

Example 16.

Solving the recurrence relation $a_k = 3a_{k-1}$ for $k=1,2,3,\ldots$ and initial condition $a_0 = 2$.

Sol :

另法： (by 6.2公式)

$$r-3=0 \Rightarrow r=3 \Rightarrow a_n = \alpha \cdot 3^n$$

$$\because a_0 = 2 = \alpha$$

$$\therefore a_n = 2 \cdot 3^n$$

Let $G(x) = a_0 + a_1 x + a_2 x^2 + \ldots = \sum_{k=0}^{\infty} a_k x^k$ be the generating function for $\{a_k\}$.

First note that $a_k x^k = 3a_{k-1} x^k$

$$\Rightarrow \sum_{k=1}^{\infty} a_k x^k = 3\sum_{k=1}^{\infty} a_{k-1} x^k = 3x\sum_{k=1}^{\infty} a_{k-1} x^{k-1} = 3x\sum_{k=0}^{\infty} a_k x^k$$

$$\Rightarrow G(x) - a_0 = 3x \cdot G(x)$$

$$\because a_0 = 2 \Rightarrow G(x) - 3x \cdot G(x) = G(x)(1-3x) = 2$$

$$\therefore G(x) = \frac{2}{1-3x} = 2 \cdot \sum_{k=0}^{\infty} (3x)^k = \sum_{k=0}^{\infty} 2 \cdot 3^k \cdot x^k$$

$$\therefore a_k = 2 \cdot 3^k$$

※Using Generating Functions to solve Recurrence Relations.

Example 17.

Suppose that a valid codeword is an n-digit number in decimal notation containing an even number of 0s. Let a_n denote the number of valid codewords of length n. Use generating functions to find an explicit formula for a_n .

Solving the recurrence relation

$a_n = 8a_{n-1}$ +$10^{n-1}$for $n=1,2,3,\ldots$ and initial condition $a_1 = 9$.

$$G(x) = \frac{1 - 9x}{(1 - 8x)(1 - 10x)}$$

Expanding the right-hand side of this equation into partial fractions (as is done in the integration of rational functions studied in calculus) gives

$$G(x) = \frac{1}{2}\left(\frac{1}{1 - 8x} + \frac{1}{1 - 10x}\right).$$

Using Example 5 twice (once with $a = 8$ and once with $a = 10$) gives

$$G(x) = \frac{1}{2}\left(\sum_{n=0}^{\infty} 8^n x^n + \sum_{n=0}^{\infty} 10^n x^n\right)$$

$$= \sum_{n=0}^{\infty} \frac{1}{2}(8^n + 10^n)x^n.$$

Consequently, we have shown that

$$a_n = \frac{1}{2}(8^n + 10^n).$$

※Proving Identities via Generating Functions

Example 18.

Use generating functions to show that

$$\sum_{k=0}^{n} C(n,k)^2 = C(2n,n)$$

Where n is a positive integer.

$$(1+x)^{2n} = ((1+x)^n)^2$$

$$= [C(n,0) + C(n,1)x + C(n,2)x^2 + \ldots + C(n,n)x^n]^2$$

作业

- § 8.1 – 10, 14
- § 8.2 – 10, 42, 46
- § 8.3 – 14, 28
- § 8.4 –8,**16,24,38**